

Experiment-2

To Solve 8 Queen Problem

Aim:

To develop a Python program to solve the **8-Queens Problem** using the **Backtracking** algorithm.

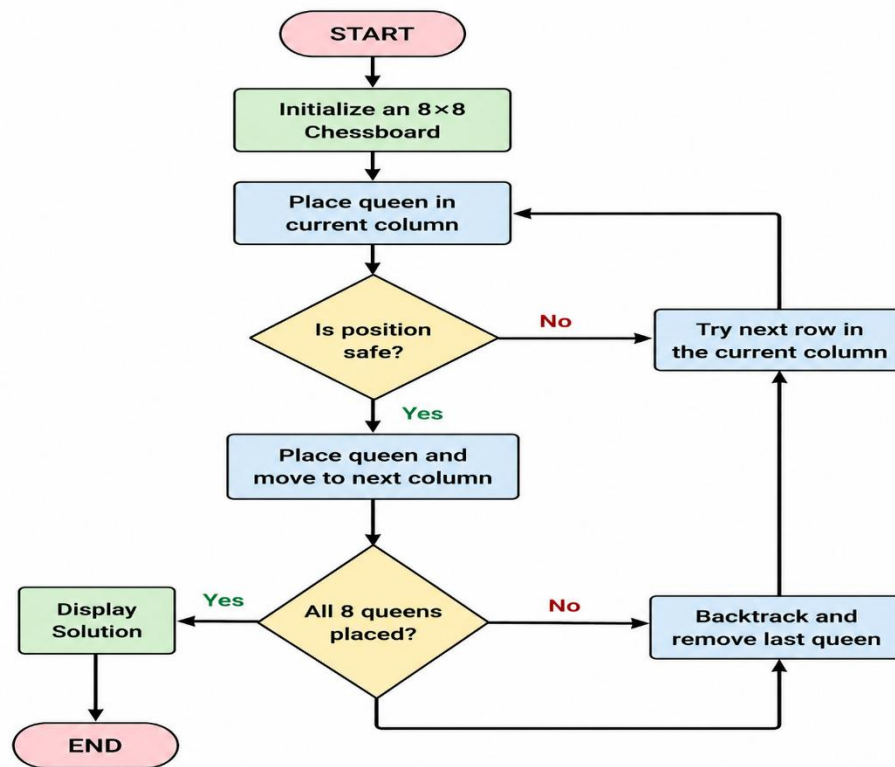
Objective

- To solve the 8-Queens problem using Backtracking.
- To place eight queens on an 8×8 chessboard without attacking each other.
- To understand constraint satisfaction problems in Artificial Intelligence.

Algorithm

1. Start the program.
2. Initialize an empty 8×8 chessboard.
3. Place a queen in the first column.
4. Check if the queen placement is safe (no two queens share the same row, column, or diagonal).
5. If safe, place the queen and move to the next column.
6. If no safe position is found, backtrack and move the previous queen.
7. Repeat until all eight queens are placed.
8. Display the final board configuration.
9. Stop the program.

Flowchart:



Code:

N = 8

```
board = [[0]*N for _ in range(N)]
```

```
def safe(row, col):
```

```
    for i in range(col):
```

```
        if board[row][i]:
```

```
            return False
```

```
    i, j = row, col
```

```
    while i >= 0 and j >= 0:
```

```
        if board[i][j]:
```

```
            return False
```

```
        i -= 1
```

```
j -= 1
```

```
i, j = row, col
```

```
while i < N and j >= 0:
```

```
    if board[i][j]:
```

```
        return False
```

```
    i += 1
```

```
    j -= 1
```

```
return True
```

```
def solve(col):
```

```
    if col == N:
```

```
        return True
```

```
    for i in range(N):
```

```
        if safe(i, col):
```

```
            board[i][col] = 1
```

```
            if solve(col + 1):
```

```
                return True
```

```
            board[i][col] = 0
```

```
return False
```

```
if solve(0):
```

```
    print("Solution:")
```

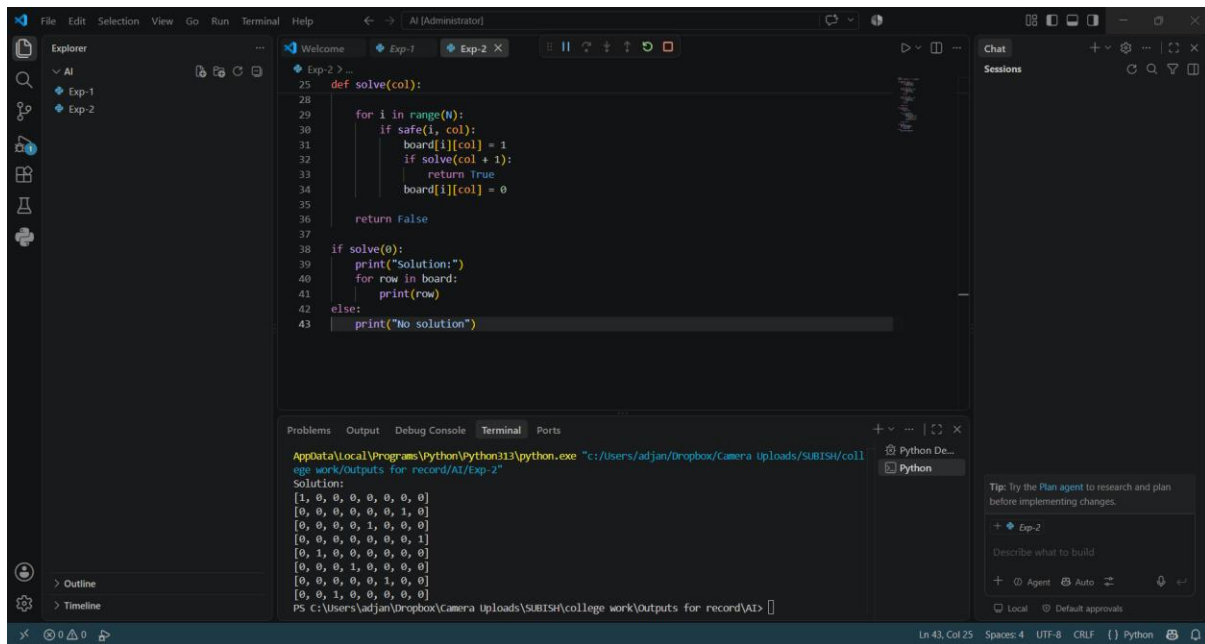
```
    for row in board:
```

```
        print(row)
```

```
else:
```

```
    print("No solution")
```

Output:



The screenshot shows a Visual Studio Code editor with a Python file named 'Exp-2.py' open. The code implements a backtracking algorithm to solve the 8-Queens problem. The terminal output shows the execution of the script, which successfully finds a solution and prints it as a list of row indices for each column.

```
25 def solve(col):
26     for i in range(N):
27         if safe(i, col):
28             board[i][col] = 1
29             if solve(col + 1):
30                 return True
31             board[i][col] = 0
32     return False
33
34 if solve(0):
35     print("Solution:")
36     for row in board:
37         print(row)
38 else:
39     print("No solution")
40
```

Terminal Output:

```
AppData\Local\Programs\Python\Python313\python.exe "C:/Users/adjan/Dropbox/Camera uploads/SUBISH/college work/Outputs for record/AI/Exp-2"
Solution:
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
```

Result:

The 8-Queens problem was successfully solved using the Backtracking algorithm, and a valid arrangement of eight queens was obtained.

Conclusion:

The Backtracking algorithm efficiently solved the 8-Queens problem by placing queens without conflicts, demonstrating its effectiveness for constraint satisfaction problems in Artificial Intelligence.